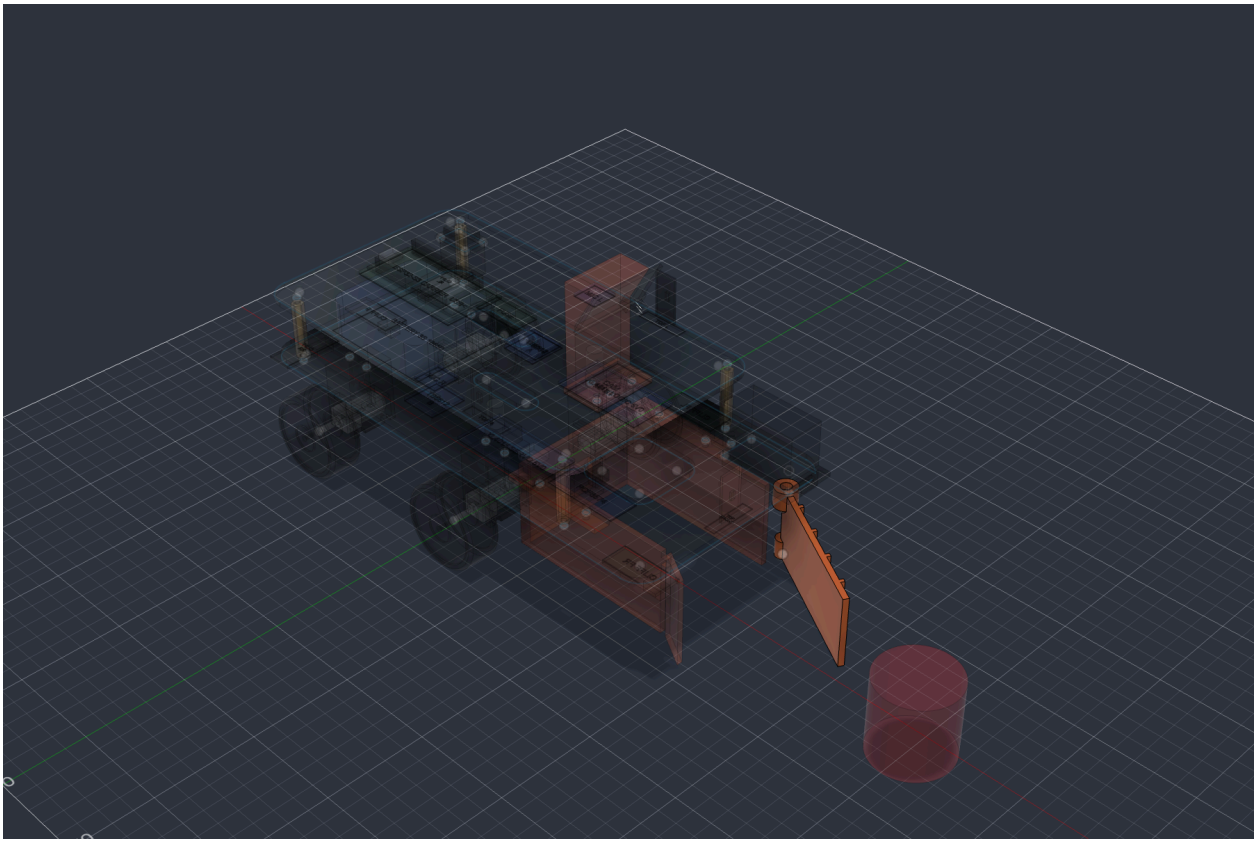
A.E.G.I.S. MK2

How A.E.G.I.S. works

The whole system explained in plain language: hearing, seeing, thinking, staying safe



Autonomous search-and-rescue rover
TL101 Tinkering Lab, IIT Ropar, AY 2026-27

This document explains the whole system in plain language: what each part does, how it does it, and why we built it that way. You do not need to read any code to follow it. Words in **bold** are explained in the glossary at the end.

1. The idea in one page

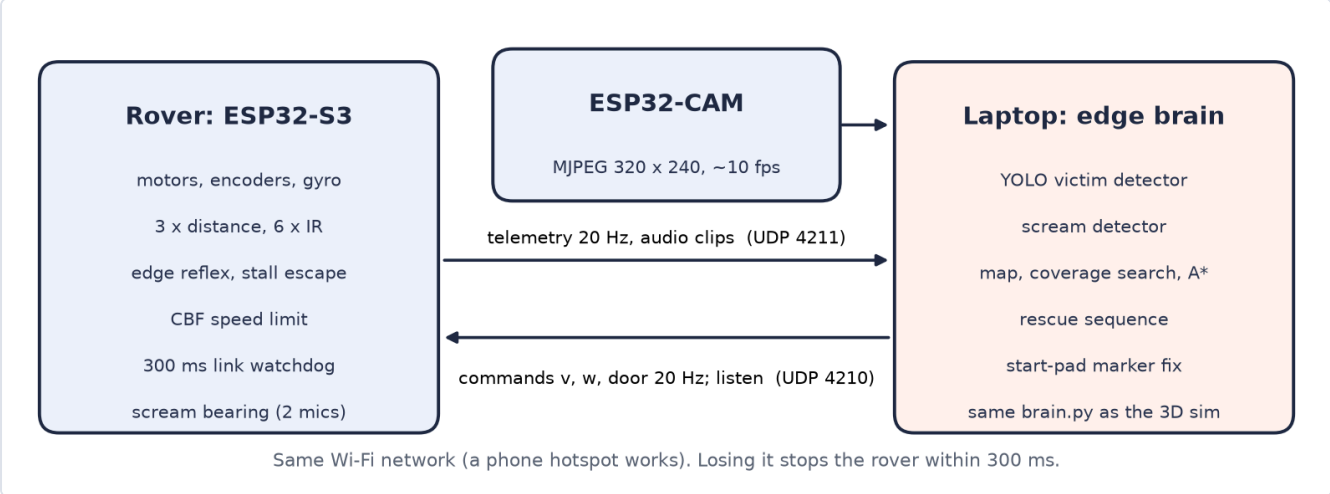
A small rover is placed on a lab table (about 1.8 x 1.2 m) with some obstacles on it, like books and boxes. Somewhere on the table is the **victim**: an orange token, 40 mm tall. The rover must:

1. **Hear** a distress call (a scream played from a phone next to the token) and work out which direction it came from.
2. **Search** the table, looking at every part of it, without bumping into things or falling off the edge.
3. **Recognise** the victim with its camera, and not be fooled by look-alike decoys (an orange cube, an orange disc, a red ball).
4. **Rescue** it: drive the token into a covered bay at the front and close a door behind it.
5. **Return** to where it started, along a path it has already driven safely.

There are two demos. In **demo 1** the token is hidden behind obstacles and there is no sound, so the rover must search carefully. In **demo 2** the phone screams, so the rover can head toward the sound first.

The original plan used an NVIDIA Jetson computer on the robot and a robot that learns by trial and error. We changed both. A Jetson alone costs more than our whole Rs 15,000 budget, and a robot that learns by crashing breaks itself. Instead, the heavy thinking runs on a laptop over Wi-Fi, and everything was first developed and tested in a computer **simulation**.

2. The big picture: three computers working together



What runs where, and how the three boards talk.

The system is split by one rule: **anything that keeps the rover safe runs on the rover itself; anything that needs a lot of computing runs on the laptop.**

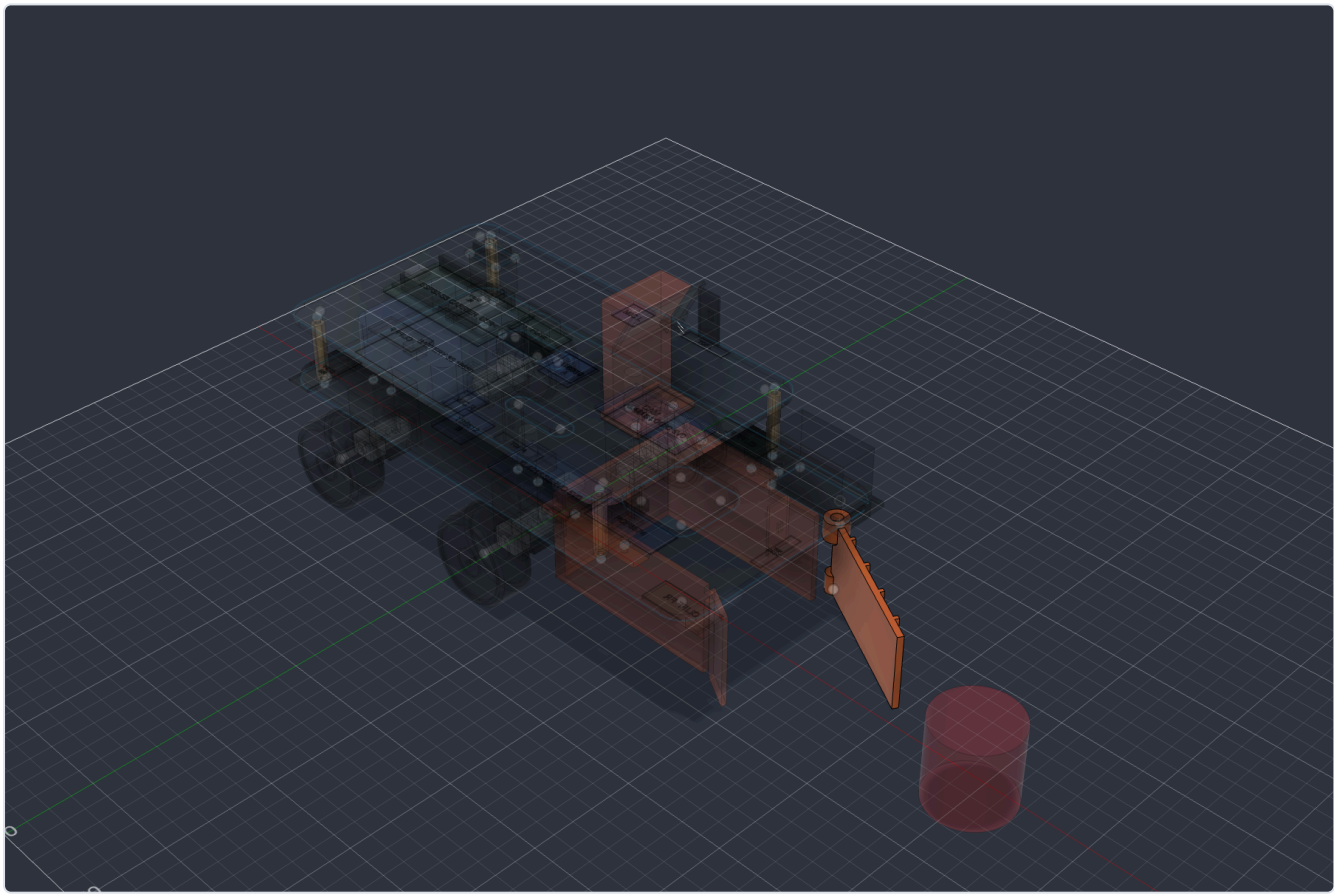
Part	What it is	Its job
Rover: ESP32-S3	A small, cheap microcontroller (two processor cores, Wi-Fi)	Drives the four wheels, reads every sensor, keeps the rover safe, works out the direction of a sound
Camera: ESP32-CAM	A second small board with a camera	Sends video to the laptop, about 10 pictures per second
Laptop: the "edge brain"	Our own Mac	Recognises the victim in the video, recognises screams, builds a map, decides where to go next

They talk over a normal Wi-Fi network (a phone hotspot works). The laptop sends driving commands ("go forward at 0.12 m/s, turn left a little, door open") 20 times a second. The rover sends back what its sensors see, 20 times a second.

Why "edge"? "Edge computing" means doing the heavy work on a computer close to the robot (our laptop, on the same Wi-Fi) instead of far away on the internet, or on an expensive computer inside the robot.

What if Wi-Fi drops? The rover stops by itself if no command arrives for 0.3 seconds. And its safety reflexes (section 6) never depend on Wi-Fi at all.

3. The body



The Mk2 rover in Autodesk Fusion, intake door open.

The rover is about 26 x 14 x 15 cm and weighs about 0.7 kg without the token. It has:

- **Two decks** of clear 3 mm acrylic, laser-cut, joined by four 30 mm metal pillars. The lower deck carries the battery, motor drivers and power parts; the upper deck carries the "brain" board, the gyro, the

microphones and the camera mast. Every part's outline is **engraved** into the deck, so you can see exactly where it goes.

- **Four-wheel drive**: four small N20 gear motors, each in a 3D-printed mount under the lower deck, each with an **encoder** that counts how far its wheel has turned.
- **The intake bay** at the front: a 3D-printed funnel that guides the token in, and a **door** swung by a servo motor that closes behind it. A small infrared sensor at the back of the bay tells the rover when the token is inside.
- **A camera mast** that holds the camera 14 cm up, tilted 30 degrees down at the table.

Everything was designed in Autodesk Fusion as a model that is generated by a script. Change a number (for example the wheel size) and the whole model rebuilds itself.

4. Power

A **3S lithium-ion battery** (three 18650 cells, 11.1 V, 2.2 Ah) powers everything. Its current goes through a **6.3 A fuse** (in case of a short circuit) and a **power switch**, then splits:

- straight to the two **motor drivers**, which need the full battery voltage for the motors;
- to two **voltage regulators** (XL4015), each turning 11 V into 5 V: one for the brain board and camera, one only for the door servo. The servo has its own regulator because when it pushes hard, it pulls a lot of current, and that could make the brain board's voltage dip and restart it.

We calculated the worst case: all four motors stalled plus the servo pushing, about 5.8 A, which is 2.7 times the battery's capacity per hour ("2.7 C"). The battery is rated for 10 C, so it is safe. The rover also measures its own battery voltage and heads home when it gets low.

5. Moving: wheels, encoders and knowing where it is

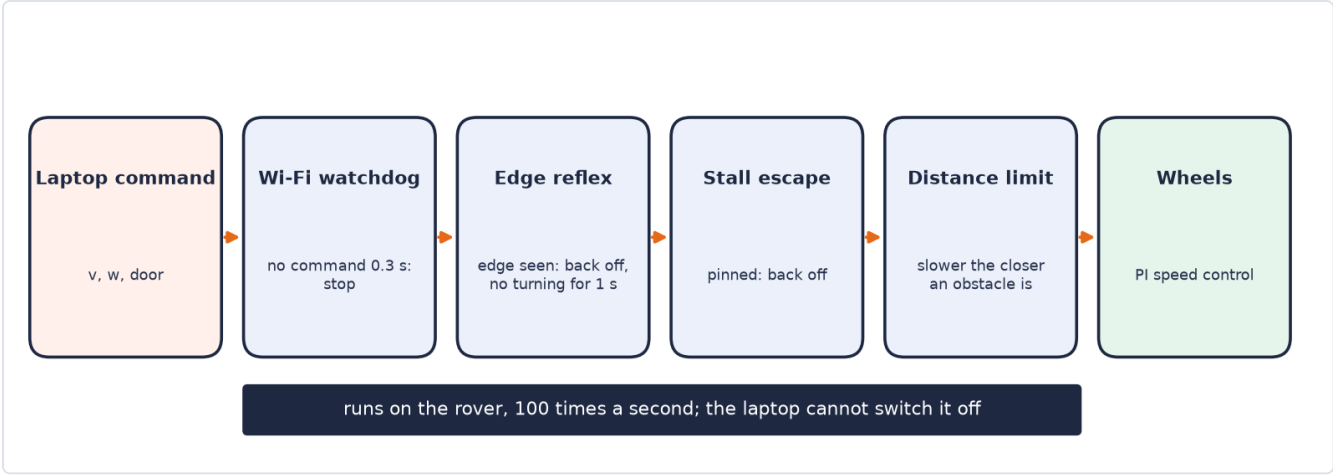
Driving. The rover steers like a tank (**skid steering**): both left wheels turn at one speed, both right wheels at another. Same speed: straight. Different speeds: a curve. Opposite directions: it spins on the spot.

Holding a speed. When the brain asks for "0.12 m/s", the rover does not just send a fixed power to the motors, because the right power changes with the battery level, the floor and the load. Instead, 100 times a second, each wheel's encoder says how fast it is really turning, and a small controller (a **PI controller**) nudges the power up or down until the real speed matches the requested one.

Knowing where it is. There is no GPS indoors. The rover works out its position by **odometry**: adding up how far the wheels have turned (for distance) and what the **gyro** says (for turning). It is like walking with your eyes closed and counting steps: very good over short distances, but small errors pile up. Two things correct that drift:

- the **table edges**: nobody has to measure the table. The first time an edge sensor finds an edge, the laptop marks that edge on its map as a wall (it assumes the edges run straight, along or across the rover's starting direction). Each later time a sensor sees the same edge, the rover knows exactly how far it is from it and corrects its position;
- the **start pad**: a printed square pattern (an **ArUco marker**) at the starting point. When the camera sees it, the laptop can calculate the rover's exact position and heading from the marker's shape in the picture, to about 1-2 cm.

6. Staying safe: reflexes that cannot be overruled

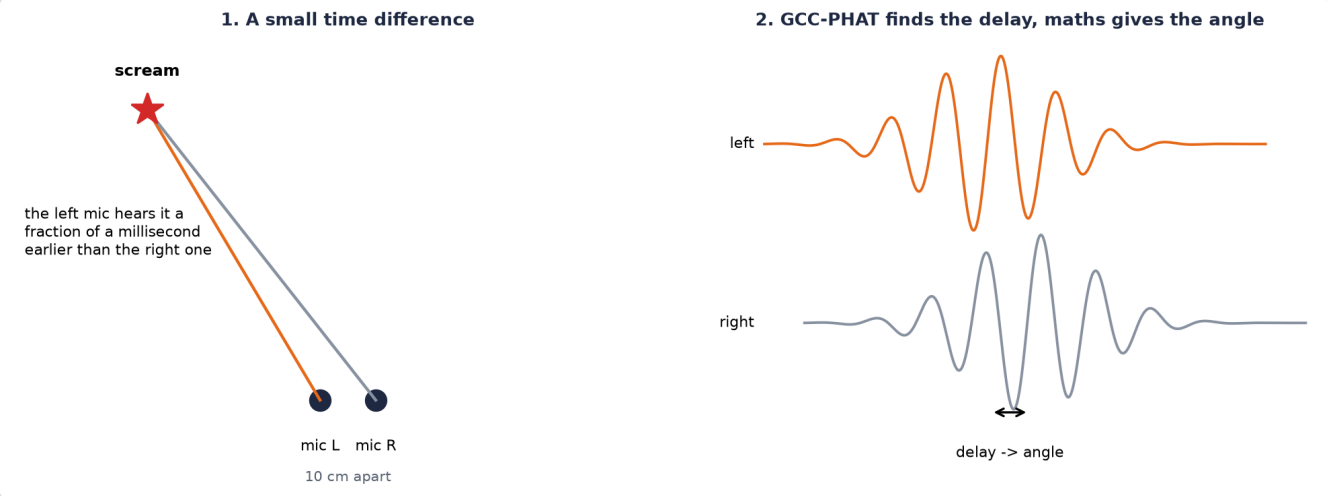


Every laptop command passes these checks on the rover before the wheels turn.

The laptop decides where to go, but the rover itself checks every command before obeying it. These checks run 100 times a second on the rover, and the laptop cannot switch them off:

1. **Table-edge reflex.** Five infrared sensors under the lower deck (front-left, front-right, rear, and both rear corners) look down at the table. Over the edge, there is no table to reflect their light. If a **front** sensor sees the edge, the rover backs off 6 cm in a straight line; if a **rear** one does, it drives forward 6 cm. For one second afterwards it may not turn, and may not move toward that edge, because turning near an edge can swing a corner off it. The rear-corner sensors were added after the simulation showed the rover falling off backwards while turning.
2. **Distance limit.** A laser distance sensor (**time-of-flight**) at the front measures how far the nearest obstacle is. The closer the obstacle, the slower the rover is allowed to drive forward, so it can never hit something in front of it at speed. (This is a **control barrier function**: a mathematical rule that keeps the rover inside a safe zone.)
3. **Stall escape.** If two or more wheels are pushing hard but barely turning (the rover is stuck against something it cannot see, such as a low object), it backs off.
4. **Wi-Fi watchdog.** No command for 0.3 s: stop.
5. **Faults.** If the rover tips over, the battery is too low, or the edge sensors stop answering, the motors switch off.

7. Hearing: which way did the scream come from?



Two microphones 10 cm apart: the arrival-time difference gives the direction.

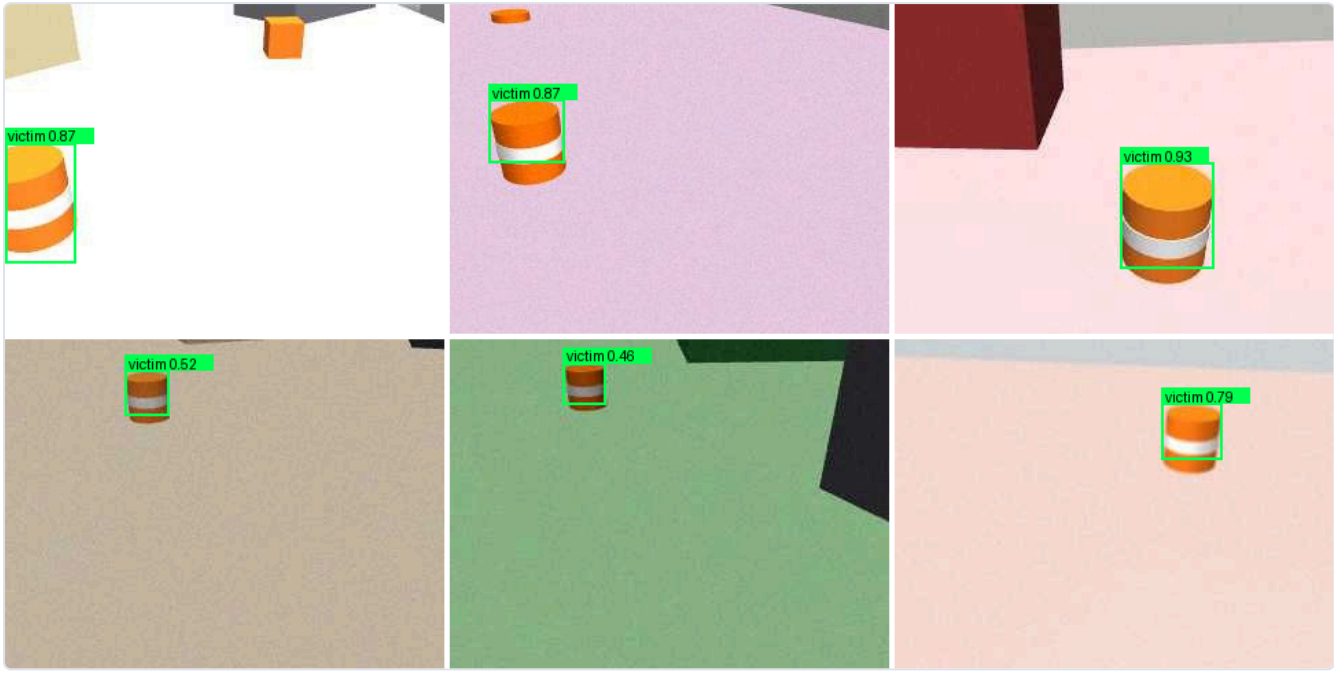
Two small digital microphones sit 10 cm apart on the upper deck. A sound coming from the left reaches the left microphone a tiny bit earlier than the right one. For a sound fully to one side, the difference is about 0.3 milliseconds, which is 14 samples at the rate the rover records (48,000 samples per second).

The rover finds that tiny delay with a method called **GCC-PHAT**: it compares the two microphone signals at every possible delay and picks the one where they line up best. From the delay and the speed of sound, it calculates the angle. In tests this is accurate to a few degrees.

The front/back problem. Two microphones cannot tell "30 degrees to the left in front" from "30 degrees to the left behind": both give the same delay. The rover solves this by turning. After a turn, the real source stays in the same place on the table while its mirror image jumps. The laptop keeps a tally of every guess, in table directions, and after a few turns only the true direction keeps getting votes.

Is it a scream? Direction alone is not enough: a fan or people talking also make sound. When the laptop asks, the rover records 0.8 s of audio and sends it over. The laptop checks it for the features of a scream (high pitch, loud, sustained). In simulation it caught 80 of 80 screams and gave 0 false alarms on 160 clips of talking and fan noise. The search only uses the direction when the sound really is a scream.

8. Seeing: finding the victim in the camera picture



What the detector sees: simulated camera frames with the victim found, decoys ignored.

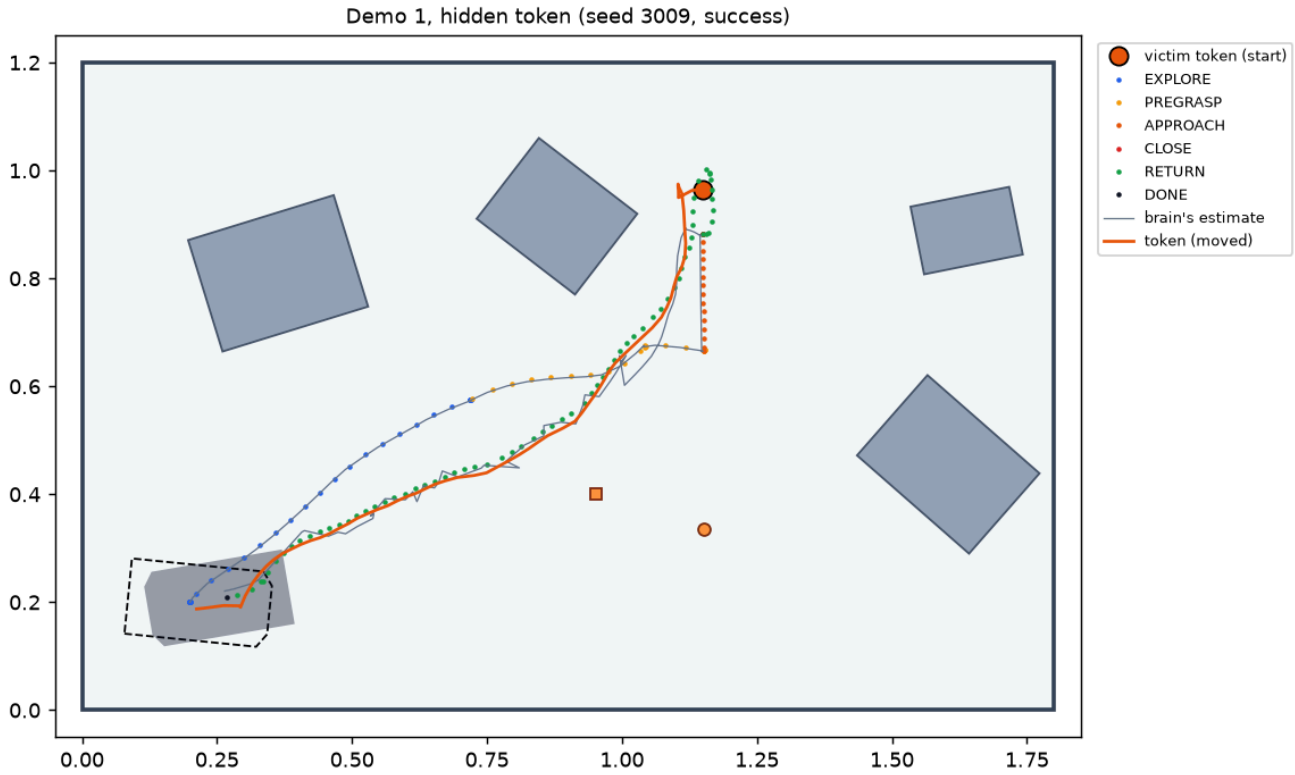
The camera's video goes to the laptop, where a neural network called **YOLO** (version 8, the small "nano" model) looks at each picture and draws a box around anything that looks like the victim token. It was trained on 4,800 pictures that the simulation rendered, each labelled automatically (the simulation knows where the token is). On 800 pictures it had never seen, it found 98.9 % of the tokens, and 98.7 % of its detections were right.

From a box in the picture to a spot on the table. The camera's height (14 cm) and tilt (30 degrees) are known. So the bottom of the box (where the token touches the table) can be traced back along the camera's line of sight down to the table surface. Combined with the rover's position at the moment the picture was taken, that gives the token's position on the table.

A real-world problem we found and fixed. The camera is a separate board, so its pictures do not carry the rover's position. Pairing each picture with the latest position put the token up to 28 cm off while the rover was turning. Now the laptop keeps the rover's positions from the last 2 seconds and uses the position from the moment the picture was taken. Pictures taken during fast turns are skipped.

The catch: the detector has only seen rendered pictures. Before the demo it must be retrained on real photos of our real token (we have a tool for collecting them).

9. Thinking: the map and the search



One simulated demo-1 mission: blue = searching, orange = approaching, green = going home; the grey line is where the rover thought it was.

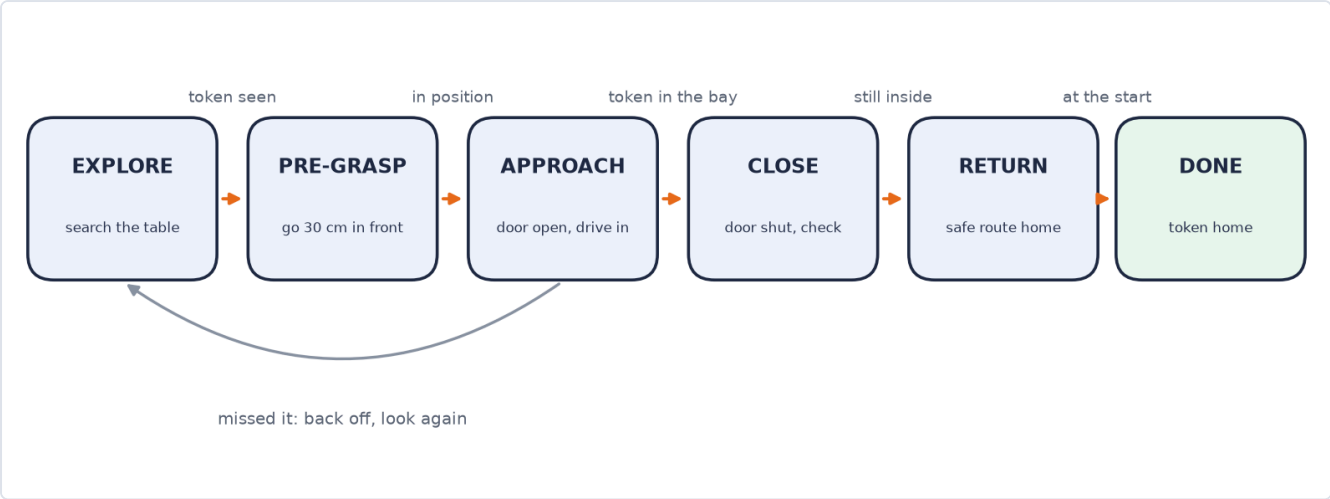
The laptop divides the table into a grid of 2 cm squares and keeps two maps:

- an **obstacle map**: which squares are blocked. The three laser distance sensors (front, left, right) mark squares as blocked or free as the rover drives.
- an **inspected map**: which squares the camera has already looked at. From the camera's position and angle, the laptop knows exactly which patch of table each picture covers.

Choosing where to go. The laptop considers many possible viewpoints and picks the one that would show the most unseen table per metre of driving. In demo 2, viewpoints in the scream's direction get a bonus, so the rover heads toward the sound but still looks around. It also remembers where it has already looked from, so it does not keep returning to the same spot.

Getting there. It plans a route around obstacles with **A*** (a standard shortest-route method, like a map app), keeping 20 cm from every table edge it has found so far, then follows the route. Until it has found an edge, the edge reflex (section 6) is what stops it there.

10. The rescue, step by step



The rescue as a state machine.

The laptop's behaviour is a **state machine**: a fixed set of modes, with clear rules for moving between them.

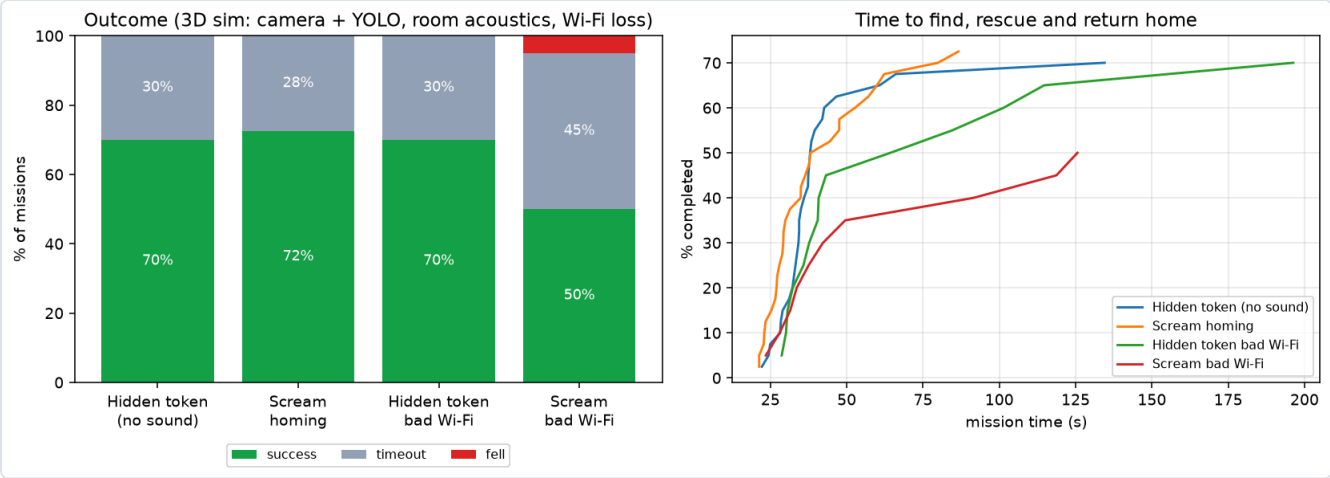
1. **Explore**: search as in section 9, until the camera has seen the token several times in the same place.
2. **Pre-grasp**: drive to a point 30 cm in front of the token, facing it.
3. **Approach**: open the door, then drive slowly straight at the token. The bay's funnel guides it in.
4. **Close**: the bay sensor sees the token inside, so close the door and check that it is still there.
5. **Return**: plan the shortest route home **using only squares the rover has already driven through**, so the way back is known to be clear. Near home the camera sees the start pad and corrects the position.
6. **Done**: back within a few centimetres of the start, token inside.

If something goes wrong (the token was missed, or there is no route), the rover backs off and tries again, and after a few failures it goes back to exploring.

11. How we tested it without building it

Everything was tested in a computer **simulation** of the lab before buying parts:

- **MuJoCo**, a physics engine used in robotics research, simulates the table, its edges, the obstacles, the token and decoys being pushed around, the rover's wheels and door, and renders what its camera would see.
- **Room acoustics** software simulates the scream bouncing around a 6 x 5 m room with a fan running.
- A **Wi-Fi model** adds delays, lost messages and short drop-outs.
- The rover's own control code (written in C) runs inside the simulation, so the simulation tests the real code, not a copy.



Outcomes over 120 simulated missions on random table layouts.

Test (random table layouts)	Missions completed (found, rescued, home)
Demo 1, hidden token, 40 runs	70 % (half finished within 35 s)
Demo 2, scream homing, 40 runs	72.5 % (half within 31 s)
Both demos on bad Wi-Fi (20 % messages lost), 40 runs	50-70 %
Falls off the table, all 120 runs	1

We also ran the real laptop program in real time against a stand-in for the rover that speaks exactly the same Wi-Fi messages as the real firmware (the "fake rover"). That found three bugs the simulation could never show (frame delays, the camera timing problem in section 8, and slow detector calls tripping the watchdog), all fixed. In that test, 9 of 16 missions succeeded, with no falls.

The simulation also told us what the rover **cannot** do: it crosses steps up to 4 mm and slopes up to 25 degrees, but a 6 mm kerb stops it. It is a table and floor robot.

12. What is still uncertain

Being honest about this is part of the project:

- **Nothing has run on real hardware yet.** The firmware builds and passes 41 automated tests, but real motors, sensors and Wi-Fi will need tuning (the bring-up steps in the deployment guide).
- **The detector and scream detector were trained on simulated data.** Both need real photos and real recordings.
- **Three parts were out of stock** when ordered (camera, microphones, wheels).
- **Real-time success is lower than in simulation** (9/16 against 14/16 on the same layouts), mainly in finding the way home without the start-pad fix; this is the next thing to improve.

13. Glossary

Word	Meaning
A*	A method for finding the shortest route on a grid map, avoiding blocked squares
ArUco marker	A printed black-and-white square pattern a camera can find and measure its position from
Control barrier function (CBF)	A rule that limits speed so the rover can never enter an unsafe zone (too close to an obstacle)
Edge computing	Doing heavy computing on a nearby computer (our laptop) instead of the robot or the internet
Encoder	A sensor on each motor that counts how far the wheel has turned
ESP32-S3 / ESP32-CAM	Small, cheap Wi-Fi microcontroller boards; the rover's brain and its camera
GCC-PHAT	A method for measuring the tiny time difference between two microphones
Gyro (MPU-6050)	A sensor that measures how fast the rover is turning
Odometry	Working out position by adding up wheel turns and turning rate
PI controller	A small feedback loop that adjusts motor power until the wheel turns at the requested speed
Simulation	A computer model of the robot and the lab, used to test everything before building
Skid steering	Steering by driving the left and right wheels at different speeds, like a tank
State machine	A fixed set of modes (explore, approach, return...) with rules for switching between them
Time-of-flight (VL53L0X)	A laser sensor that measures distance by timing how long light takes to bounce back
UDP	The simple, fast kind of network message the rover and laptop use
YOLO	A neural network that finds objects in pictures and draws boxes around them